

Part2:Implementation

08/01/2025

95/100 Points

Attempt 1



Review Feedback

08/01/2025

Attempt 1 Score:

95/100

View feedback

Anonymous grading: **no****Unlimited Attempts Allowed**

15/01/2025

Details**Part 2: Implementation - Functional application, design patterns, GUI**

Weighting %:	50	Submission Deadline:	8th January 2025
Authorship:	Individual	Target date for returning marked coursework:	28th Jan 2025
Tutor setting the work:	Dr John Kanyaru	Number of hours you are expected to work on this assignment:	50

This **Part 2** of the assignment assesses the following module Learning Outcomes (from Definitive Module Document):

1. Be able to define software architecture.
1. Have knowledge and understanding of common software architecture patterns, Conway's law and its implications.
1. Understand the advantages of a three-tier architecture.
1. Be able to apply common software architecture patterns to design systems and to describe a system architecture using common modelling notation. ✓
1. Be able to implement a system using a three-tier architecture. ✓
1. Be able to use the Model-View-Controller pattern to design a user interface. ✓

Considerations for implementation

This part 2 of your assignment builds on the design models you produced in Part 1. Hence, you should refer to the scenario in Part 1 and your models.

The main feature of your application is to demonstrate a depot worker processing customers in the queue collecting their parcels. For purposes of a common denominator set of classes, your program should contain at least the following classes:

Parcel class - to contain the structure (attributes & methods) of a parcel

Customer class - to the blueprint of customer objects

QueueofCustomers class - to enable maintaining a queue/list of customer of objects

ParcelMap class - uses a map data structure to store parcel objects.

Log class - to create a single log instance for containing log of events and related data. You can use the inbuilt StringBuffer class to add these events to an instance of it.

Worker class - to contain the logic for a worker processing a customer, calculating the fee, and releasing the parcel;

Manager class - this is your driver class where you instantiate QueueofCustomers, ParcelMap and your GUI. This is where you ideally read in your data files using suitable methods.

Again, develop your application iteratively, starting with console i/o before progressing to building a GUI. Once you have the above classes setup with suitable details (attributes, methods), commit them to your git repository with a suitable message (e.g., class structure version 1). Then do check if they function as expected using console i/o. You should choose the most appropriate data structures (chosen from lists, maps, sets) for maintaining collections of your objects. When making your decision, imagine that you have a large number of parcels and customers.

Here is a sample dataset that you can use as input if you wish, or you can come up with your own data files to check that your program is working correctly. Sample customer data is [here](https://herts.instructure.com/courses/116994/files/9894367?wrap=1)

(<https://herts.instructure.com/courses/116994/files/9894367?wrap=1>)_ ↓

(https://herts.instructure.com/courses/116994/files/9894367/download?download_frd=1) and sample

parcel data is [here](https://herts.instructure.com/courses/116994/files/9894369?wrap=1) (<https://herts.instructure.com/courses/116994/files/9894369?wrap=1>)_ ↓

(https://herts.instructure.com/courses/116994/files/9894369/download?download_frd=1) .

Your classes should have suitable instance variables and methods. The calculation of the fee must be in a **calculateFee** method so that you can test it easily. This method can be in either the Worker or Manager class.

The GUI should show the list of parcels still to be processed, the current queue of customers, and the details of the parcel currently being processed by the worker. Each panel should be updated whenever the data which is displayed in that panel is altered. Do several git commits as you build your GUI. For example, when you build a GUI to display Parcels, commit your new code version to git, do the same when you get to display customers and calculate fees. You should ensure to have at least 4 git commits over the period of your implementation.

In your implementation, ensure to use the Singleton pattern in a Log class which is used to record every event (i.e. customer joins queue, customer is removed from queue, processing details are displayed etc.) The Log file is finally written to a text file and should be easy to read. Additionally, you should use the MVC pattern to structure your GUI components. The GUI must be built using

Swing components (buttons, panels, Frames, etc) to support responsive use of the features of your system. You must not use JavaFX or GUI generators. All GUI code must be written by yourself.

You should write a reflective report of between 400 and 500 words describing how your models in part 1 are realised in your implementation (part 2). An example would be - **use case A** is implemented in class **Worker** as method **do_counts()**. Class K in my class model is implemented as class K in my implementation and is related to class T using a bidirectional relationship as depicted in the class diagram, etc... Your report should also reflect on how you have used the MVC pattern to structure your application.

Your work for part 2 will be marked using the following guide:

Functionality:

15 Accurate processing (loading of parcels & customers data files, fee calculation; sorting of parcels, adding new customers to the queue, removing customers, adding new parcels, marking parcel as processed)

10 Contents of output text file (log file)

Design patterns:

10 Use of singleton pattern in logging

15 Use of MVC Pattern in the GUI

GUI:

10 Displays customers, parcels, & displays current parcel selected for processing

10 Functional & interactive

Class structure:

5 Classes have expected details (methods, attributes)

10 Use of suitable relationships (e.g., association, composition) among classes

Git log & report

5 A screenshot of git log commits that show application was developed iteratively over time

10 Reflective report (400 to 500 words) on how designs in part 1 are reflected in the Implementation, and how MVC pattern has been used to structure your application

Submission Requirements for part 2:

Submit your .java files as a zip file. The code will be checked that it works and that it has been written by yourself to address the requirements of this assignment. Include a screenshot of your git log, and your (400 - 500 words) report. Your zip file should be named using this pattern:

your_student_number.surname_part2.zip.

Type of Feedback to be given for this assignment:

Feedback will include scores for each part as outlined above and comments from the marker.

Additional information:

- Regulations governing assessment offences including Plagiarism and Collusion are available from https://www.herts.ac.uk/_data/assets/pdf_file/0007/237625/AS14-Apx3-Academic-Misconduct.pdf  (https://www.herts.ac.uk/_data/assets/pdf_file/0007/237625/AS14-Apx3-Academic-Misconduct.pdf)

Links to an external site.  (https://www.herts.ac.uk/_data/assets/pdf_file/0007/237625/AS14-Apx3-Academic-Misconduct.pdf)

(UPR AS14).

- Guidance on avoiding plagiarism can be found here: <https://herts.instructure.com/courses/61421> (<https://herts.instructure.com/courses/61421>) (see the **Referencing** section)
- For **undergraduate modules**:
 - a score of 40% or above represents a pass performance at honours level.
 - late submission of any item of coursework for each day or part thereof (or for hard copy submission only, working day or part thereof) for up to five days after the published deadline, coursework relating to modules at Levels 0, 4, 5, 6 submitted late (including deferred coursework, but with the exception of referred coursework), will have the numeric grade reduced by 10 grade points until or unless the numeric grade reaches or is 40. Where the numeric grade awarded for the assessment is less than 40, no lateness penalty will be applied.

▼ View Rubric

Select Grader

John Kanyaru (Teacher)

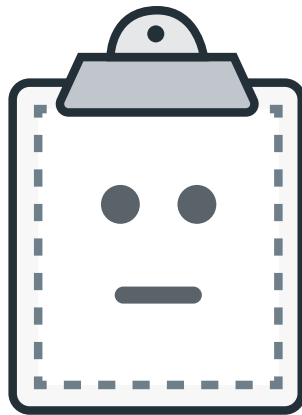


Design Implementation

Criteria	Ratings	Points
Class Structure view longer description	Comments Code structure is good, written well and class associations are fine	15 / 15 pts

Design Implementation

Criteria	Ratings	Points
Functionality view longer description	Comments The application functions well, loads files well, and does generate reports of processed parcels, also supports searching; needed to do some sorting also; overall, comprehensive!	23 / 25 pts
Responsive GUI view longer description	Comments The GUI is good, responsive and functional	20 / 20 pts
Use of Design Patterns view longer description	Comments Excellent use of the singleton and MVC patterns	25 / 25 pts
Git log view longer description	Comments Git log is submitted	5 / 5 pts
Reflective report view longer description	Comments The discussion on use cases seems to be about architecture; other parts of the report are fine	7 / 10 pts
		Total points: 95



Preview Unavailable

21079043.Desai_part2.zip

↓ [Download](#)

(https://herts.instructure.com/files/10367182/download?download_frd=1&verifier=NyF6k6UHaHpbgEI1r8cOUTtGEIhM5FPH45JSYbEo)